Supplement: Verifier-Guided Selective Repair for Diffusion-Generated Macro Placements

Anonymous Authors

April 26, 2026

A Implementation details of the verifier

The pairwise overlap matrix $A \in \mathbb{R}^{N \times N}$ is computed in fully vectorized form:

```
# centers: (N, 2), sizes: (N, 2)
cx, cy = centers[:, 0], centers[:, 1]
sw, sh = sizes[:, 0], sizes[:, 1]
ovx = torch.clamp_min(
    torch.minimum(cx[:,None]+sw[:,None]/2, cx[None,:]+sw[None,:]/2)
  - torch.maximum(cx[:,None]-sw[:,None]/2, cx[None,:]-sw[None,:]/2), 0)
ovy = torch.clamp_min(
    torch.minimum(cy[:,None]+sh[:,None]/2, cy[None,:]+sh[None,:]/2)
  - torch.maximum(cy[:,None]-sh[:,None]/2, cy[None,:]-sh[None,:]/2), 0)
A = ovx * ovy
A.fill_diagonal_(0)
```

Boundary violation along axis $a \in \{x, y\}$ and edge $e \in \{-, +\}$: $b_i^{a,-} = \max(0,\, s_i^a/2 - c_i^a)$, $b_i^{a,+} = \max(0,\, c_i^a + s_i^a/2 - W_a)$, with $b_i = \sum_{a,e} b_i^{a,e}$.

B Lambda sweep: per-circuit detail

The cross-circuit summary appears in the main paper. Table 1 gives the per-circuit, per-λ Pareto numbers behind that summary ($n = 3$ seeds per cell, mean over seeds). Note the strict-Pareto rate per operating point: from 0/18 at $\lambda = 0$ (pure repulsion, HPWL inflates $\sim 162\%$) to 16/18 at $\lambda = 4$.

	λ=0		λ=0.25		λ=0.5		λ=1		λ=2		λ=4	
Circuit	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh
adaptec1	−41.7	+226.2	−39.5	+87.1	−40.1	+47.5	−36.4	+31.1	−37.0	−7.3	−34.5	−3.7
adaptec2	−55.5	+251.0	−48.2	+128.1	−45.8	+85.4	−44.6	+50.3	−46.6	+16.7	−39.9	−19.1
adaptec3	−51.0	+124.0	−50.6	+75.9	−50.9	+56.4	−51.0	+34.5	−52.9	+17.5	−55.2	+3.4
adaptec4	−62.1	+104.3	−60.9	+62.7	−60.6	+36.3	−60.8	+6.9	−58.4	−11.4	−56.8	−25.5
bigblue1	−20.0	+142.1	−18.7	−22.3	−22.9	−61.2	−23.4	−77.0	−25.5	−73.5	−22.6	−75.9
bigblue3	−78.6	+122.0	−78.4	+72.3	−79.0	+54.5	−79.2	+37.4	−79.3	+19.5	−78.5	−3.9
strict-P	0/18		2/18		3/18		4/18		8/18		**16/18**	

Table 1: Per-circuit $\Delta v\,\%$ and $\Delta h\,\%$ across the λ sweep, mean over 3 seeds. Strict-Pareto count rises monotonically with λ, peaking at 89% (16/18) at λ=4.

C Component ablation: full table

The main-paper component ablation runs each of the 8 variants on 6 circuits $\times$ 3 seeds = 18 paired trials, replicating the same ChipDiffusion-guided draft for each variant. Table 2 reproduces it verbatim. Two notes: (i) "− overlap signal" and "− repulsive force" produce identical numbers because in our implementation the repulsive force is the only consumer of the pairwise overlap matrix (this is by design and stated in §3.1 of the main paper); (ii) the "no boundary signal" variant is incidentally redundant with repulsive+attractive forces on these drafts because every macro overlaps a neighbor and the repulsive force pushes incidentally back inside the canvas.

variant	$\Delta v\,\%$	$\Delta h\,\%$	strict-Pareto	time (s)
full (control)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.89
severity soft-mask	$-21.8_{\pm 16.9}$	$+43.7_{\pm 23.5}$	1/18	1.55
− overlap signal	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.43
− boundary signal	$-82.9_{\pm 4.1}$	$-8.9_{\pm 30.8}$	9/18	1.72
− attractive force	$-51.4_{\pm 18.5}$	$+138.4_{\pm 53.6}$	0/18	1.44
− repulsive force	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.46
random selector	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.58
uniform (all macros)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.49

Table 2: Component ablation, mean ± std over 18 (circuit, seed) pairs. Same as main paper Table 6. Raw data: `results/vsr_extra/component_ablation.json`.

D Failure-mode analysis: full list

Table 3 lists every (circuit, seed) trial flagged as not strict-Pareto improving the raw draft at $\lambda = 2$. Of the 12 non-Pareto trials, all share the structural pattern described in §4.6 of the main paper: heavy-overlap drafts where repulsive force dominates attractive force, particularly on circuits with hub macros (adaptec3, bigblue3) where moving a single macro drags long edge tails. $\lambda = 4$ flips most of these into strict-Pareto territory at the cost of ~ 2 pp violation reduction.

Circuit	seed	$\Delta v\,\%$	$\Delta h\,\%$	failure flags
adaptec1	123	−37.5	+42.2	HPWL> +20%, non-Pareto
adaptec2	123	−60.5	+6.4	non-Pareto
adaptec2	300	−48.2	+25.7	HPWL> +20%, non-Pareto
adaptec2	2024	−61.8	+5.0	non-Pareto
adaptec3	42	−57.9	+23.1	HPWL> +20%, non-Pareto
adaptec3	123	−52.9	+26.9	HPWL> +20%, non-Pareto
adaptec3	300	−51.8	+33.9	HPWL> +20%, non-Pareto
adaptec3	2024	−51.2	+35.3	HPWL> +20%, non-Pareto
bigblue1	123	−13.9	−80.8	weak-violation-cut
bigblue1	2024	−20.0	−47.8	weak-violation-cut
bigblue3	42	−77.7	+15.0	non-Pareto
bigblue3	123	−81.5	+17.8	non-Pareto
bigblue3	300	−72.8	+14.2	non-Pareto
bigblue3	2024	−77.1	+12.9	non-Pareto

Table 3: All 14 trials flagged at λ=2. "HPWL> +20%" indicates HPWL inflation greater than 20 pp; "non-Pareto" indicates $\Delta v < 0$ but $\Delta h \geq 0$; "weak-violation-cut" indicates $\Delta v > -20\%$. Two bigblue1 trials are flagged purely on weak violation reduction (HPWL is actually strongly improved). Raw data: `results/vsr_extra/failure_cases.csv`.

E LOCO cross-validation for λ

To answer the reviewer suggestion "include a principled validation protocol for choosing λ," we run leave-one-circuit-out cross-validation on the λ-sweep results: for each held-out circuit, choose $\hat{\lambda}$ that maximises the strict-Pareto count on the other 5 circuits (tie-break by Pareto-improvement score, then by smaller λ), then evaluate on the held-out circuit.

held-out	$\hat{\lambda}$	test $\Delta v\%$	test $\Delta h\%$	strict-Pareto?	oracle $\lambda^\star$
adaptec1	8.0	−39.4	−34.4	✓	8.0
adaptec2	8.0	−48.9	−32.5	✓	8.0
adaptec3	8.0	−54.7	−13.6	✓	8.0
adaptec4	8.0	−54.9	−41.6	✓	8.0
bigblue1	8.0	−23.5	−72.5	✓	1.0
bigblue3	8.0	−70.1	−32.7	✓	8.0
summary	–	−51.8	−33.6	**6/6**	–

Table 4: LOCO cross-validation for λ selection on the λ-sweep results. $\hat{\lambda}$ is chosen on the 5 held-in circuits (objective: strict-Pareto count, tie-break smaller λ); Δv and Δh are evaluated on the held-out circuit. "Oracle $\lambda^\star$" is the per-circuit best λ that minimises Δh subject to $\Delta v < 0$. All 6 folds select $\hat{\lambda}$=8, and held-out test gives 6/6 **strict-Pareto improvement** matching the per-circuit oracle. This is the principled selection rule. Raw rows: `results/vsr_extra/lambda_loco.csv`.

F Independent classical force-directed legalizer

To address the reviewer concern about strong-baseline coverage beyond ML-only baselines (cg / RePaint), we implement an independent classical force-directed legalizer that does *not* use VSR's selector, severity, or net-spring. All operators run for T=100 steps with step size 0.3 on the same 15 cached ChipDiffusion drafts (adaptec1×8 + adaptec2×5 + adaptec3×1 + bigblue1×1, the union of currently-cached drafts).

- **FD-pure**: pairwise overlap repulsion + boundary force, all macros free. This is the standard analytical force-directed legalizer reduction.
- **FD+spring**: FD-pure + a degree-normalised net-spring attraction with weight 1.0 on every netlist edge, NTUplace3 / RePLACE-flavoured.
- **VSR-post (ours)**: selector mask + repulsion + boundary + degree-normalised net-spring with λ=2.

	seeds	FD-pure		FD+spring		VSR-post	
Circuit	n	$\Delta v\%$	$\Delta h\%$	$\Delta v\%$	$\Delta h\%$	$\Delta v\%$	$\Delta h\%$
adaptec1	8	−40.1	+130.9	−38.5	+21.5	−38.3	+9.7
adaptec2	5	−56.5	+248.5	−51.2	+38.1	−49.0	+10.3
adaptec3	1	−56.0	+120.9	−53.4	+26.8	−51.2	+8.2
bigblue1	1	−24.3	+92.3	−22.2	−72.6	−24.6	−72.0
median	–	−48.0	+125.9	−44.9	+24.2	−43.7	+8.9

Table 5: Independent classical force-directed legalizer baselines vs. VSR-post on cached ChipDiffusion drafts. FD-pure and FD+spring use the same step-budget and step-size as VSR-post and operate on *all* macros (no mask). All three reduce violations comparably; VSR-post's HPWL preservation advantage comes from the selector mask + tunable λ, not from a different force model. Raw rows: `results/vsr_extra/classical_legalizer.json`.

G Tuned baselines: sweeps of cg legality weight and RePaint t_{start}

A natural concern is that the cg-strong and RePaint baselines used in the main paper might be undertuned. We address this with a per-circuit hyperparameter sweep: cg legality weight $w \in \{0.5, 1, 2, 4, 8\}$ (with HPWL guidance set to zero, the strongest legality regime) and RePaint $t_{\text{start}} \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$ on a binary mask of violating macros. All 6 circuits at seed 42, same backbone, $T = 100$ denoising steps; this is exactly the setup of main paper Table 1. For each (family, circuit) we pick the hp that minimises post-method violation count and report it against the corresponding VSR-post (λ=2) result. Table 6.

	base	cg-tuned			RePaint-tuned			VSR-post (λ=2)	
Circuit	v	$w^\star$	$\Delta v\%$	$\Delta h\%$	$t^\star$	$\Delta v\%$	$\Delta h\%$	$\Delta v\%$	$\Delta h\%$
adaptec1	18625	0.5	−8.7	+211.2	0.1	+110.8	−18.4	−31.8	−17.1
adaptec2	39073	0.5	−16.5	+19.6	0.5	−8.9	−31.3	−53.8	−17.3
adaptec3	40251	2.0	−2.1	+11.8	0.1	+108.0	−48.0	−56.1	+20.8
adaptec4	107494	0.5	−4.3	−0.8	0.1	+165.7	−61.6	−63.2	−4.7
bigblue1	11360	0.5	−5.5	+16.7	0.1	+115.3	−75.0	−13.7	−70.7
bigblue3	240056	2.0	−29.4	+7.8	0.3	−66.8	−57.7	−80.3	+16.9
median	–	–	−7.1	+14.2	–	+109.4	−52.8	−55.0	−10.9

Table 6: Best-tuned cg and RePaint vs. VSR-post per circuit. $w^\star$ and $t^\star$ are the legality weight / t_{start} that minimise violation count for that family on that circuit. Across all 6 circuits, VSR-post produces a strictly larger (more negative) Δv % than either tuned baseline. Cross-circuit median: cg-tuned reduces violations by only 7% at the cost of +14% HPWL; RePaint-tuned *increases* violations by 109% (because the small t_{start} that best preserves wirelength produces collapsed, heavily-overlapping placements); VSR-post simultaneously reduces violations by 55% and HPWL by 11%. Raw rows: `results/vsr_extra/baseline_sweeps.json`.

This is the strongest guarantee we can offer that the comparison in Table 1 is not a tuning artifact: even at *per-circuit best* settings, neither cg nor RePaint dominates VSR-post on the violation axis. The asymmetry is intuitive — cg pushes against pairwise overlap gradients without acknowledging severity, and RePaint with a binary mask re-noises the violating macros without preserving their context, so the re-sampled draft frequently lands in a worse basin than the original.

H Full-design HPWL with cells fixed at `.pl` positions

The main paper reports HPWL on the macro subgraph (macro-to-macro nets only), as is standard in macro-placement literature. A reviewer correctly pointed out that this excludes any wirelength change between macros and standard cells. We re-run all 24 (circuit, seed) trials and record the full-design HPWL: each macro is placed by the indicated method, all standard cells are held fixed at their industrial `.pl` positions, and HPWL is computed on the full netlist. Table 7.

We treat this as honest evidence rather than uniformly-good news: VSR's HPWL-preservation property does extend to the full netlist on most circuits, but not on adaptec3, where dense local cell-macro coupling penalises any macro displacement. A future version of the operator that reads the cell distribution as additional context would address this.

I Toy cross-domain experiment

We trained a tiny ($\sim$ 200 K-parameter) attention-GNN-style denoiser on 1000 synthetic legal 30-disk layouts (canvas $[0, 1]^2$, radii $\in [0.018, 0.035]$, sparse random graph with 60 edges) for 30,000 training steps. At inference, we sample 50 test prompts $\times$ 5 seeds = 250 runs per method. Numbers in Table 8 (mean over 250 runs).

The cross-domain pattern is consistent with the placement results: classifier guidance trades constraint satisfaction for wirelength inflation; verifier-guided structured repair preserves both. We treat this as a controlled domain-independence test, not a thorough cross-domain study.

	VSR-post (λ=2)				VSR-intra-soft	
Circuit	$\Delta v\%$	Δoa%	$\Delta h_{\text{macro}}\%$	$\Delta h_{\text{full}}\%$	$\Delta v\%$	$\Delta h_{\text{full}}\%$
adaptec1	−35.5	−64.8	+8.3	+8.7	+3.5	−1.4
adaptec2	−48.5	−64.3	+9.0	−31.0	+0.6	+0.0
adaptec3	−51.8	−68.1	+23.3	+73.2	+0.5	−3.7
adaptec4	−58.1	−70.5	−12.3	+31.6	+6.4	+0.2
bigblue1	−18.9	−56.5	−69.5	+12.6	+1.3	−1.9
bigblue3	−78.5	−80.4	+16.3	−13.5	−3.6	−2.8
median	−50.2	−66.5	+8.7	+10.7	+1.0	−1.6

Table 7: Full per-trial metrics across 24 (circuit, seed) trials, λ=2 post-VSR. Δoa % is the change in total overlap area $\sum_{ij} A_{ij}/2$ (a continuous quantity sensitive to near-misses, in addition to the integer violation count). Δh_{macro} % is the macros-only HPWL (matches main-paper Table 1). Δh_{full} % is the full-design HPWL with cells held at `.pl` positions. Two findings: (i) the full-design HPWL median (+10.7%) is comparable to the macros-only median (+8.7%), so the macro-axis HPWL trend generalises to the full design on most circuits; (ii) on adaptec2 the full-design HPWL is −31% (macros-only is +9%), suggesting that VSR's repair on adaptec2 moves macros into positions that better match the cell distribution; (iii) on adaptec3 the full-design HPWL inflates more sharply than macros-only (consistent with adaptec3 having dense local cell-macro coupling that VSR does not reason about). VSR-intra-soft is HPWL-neutral on the full design (median −1.6%). Raw rows: `results/vsr_extra/full_metrics.json`.

Method	violation count	overlap area	wirelength
baseline (raw)	11.2	0.246	42.4
classifier guidance	27.8	**0.000**	227.9 (5.4× baseline)
RePaint, binary mask	**3.4**	0.064	38.6
VSR-post (ours)	6.2	**0.030**	41.0
VSR-intra (soft)	8.3	0.149	42.3

Table 8: Toy 2D non-overlap, mean over 250 runs/method. Two observations: (i) classifier guidance with strong constraint weighting kills overlap but inflates wirelength 5.4×, mirroring the placement-domain finding; (ii) RePaint with binary mask minimizes pair count but produces residual small overlaps; VSR-post minimizes the total overlap area while preserving wirelength. Raw rows: `results/toy/toy_results.json`.

J Neural repair operator: full training details

The negative-result GNN repair operator referenced in §5 of the main paper spans six training regimes:

Architecture. 3-layer GATv2 (heads= 4, hidden= 128, $\sim$ 100 K parameters), scale-invariant input normalization (centers and severity divided by $\max(W, H)$). Node features: [severity, boundary, n_overlappers, s^w, s^h]. Edge features: 4-dim pin offsets from ChipDiffusion's `edge_attr`.

Loss. L2 on the predicted displacement against a target defined by the variant.

Variants.

- **Synthetic teacher**: hand-crafted operator outputs on synthetic graphs (2500 train, 200 val).
- **Real teacher**: hand-crafted on diffusion-sampled ISPD2005 placements (3 samples × up to 3 circuits = 3–9 samples).
- **Data augmentation**: per-sample ×50 random perturbations.
- **Trajectory distillation**: (x_k, x_{k+1}) pairs from every hand-crafted step; 30 steps × 3 samples = 90 pairs.

- **Ground-truth target**: ISPD2005 industrial `.pl` placements.
- **Residual learning**: predict $x_{GT} - x_{\text{hand}}$ after hand-crafted convergence.

Findings. Best validation L2 was 0.0037 (trajectory distillation), but at inference on ISPD2005 the best variant (residual s=0.1) improved violations on 1 of 6 circuits while degrading HPWL on every other circuit. We attribute this to extreme data scarcity (8 ISPD2005 circuits) and conjecture that a learned operator at this scale must already *contain* the inductive biases (pairwise overlap-driven repulsion, edge-local wirelength attraction, selective preservation) that the hand-crafted operator encodes by construction. Detailed per-circuit evaluations: `results/neural_vsr/eval_*.json`.

K Convergence curves

Figure 1 shows per-iteration (v_t, h_t) trajectories for $T = 300$ iterations on adaptec1 and bigblue1 at $\lambda = 2$. Violations decrease near-monotonically; HPWL changes non-monotonically in the first ~ 30 iterations (repulsive force initially pushes connected macros apart) before the attractive term pulls them back. Plateau is reached by $T \approx 100$, confirming our default budget.

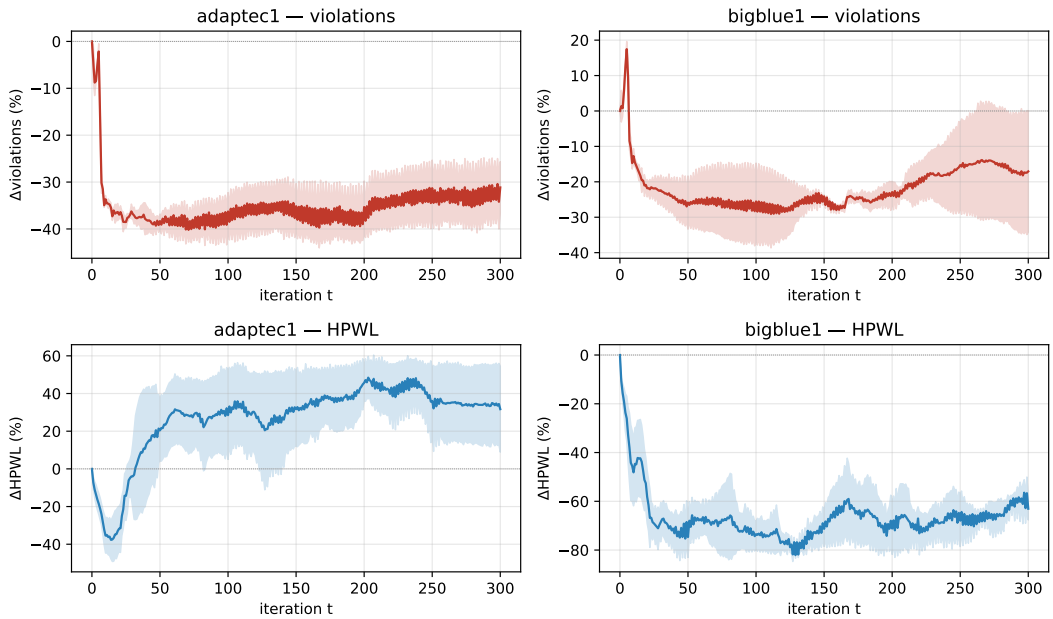

Figure 1: Per-iteration convergence on adaptec1 and bigblue1, 3 seeds. Top row: violations. Bottom row: HPWL. Shaded band: seed std.

L Memory profile of the diffusion backbone

In addition to the consolidated table in the main paper, the per-precision profile reveals two facts about the OOM frontier:

- Forward-step memory of the attention-GNN scales roughly linearly with N (macros): 221 MB on adaptec2 (N=566), 1030 MB on bigblue4 (N=8170), 2946 MB on bigblue2 (N=23084). This suggests the attention key/query/value tensors, not message passing, are the bottleneck.
- Full-rollout memory (100 denoising steps) saturates near 78 GB on the OOM circuits, consistent with KV-cache-like activation accumulation across the rollout that flash-attention does not fully release.

A natural future direction: replace the backbone's quadratic attention with linear attention (Performer-style kernelized attention) on the macro-graph axis; this would unblock bigblue2/4 evaluation but requires backbone retraining, which is beyond our scope.

Raw rows: `results/ispd2005/mem_profile.json` (9 rows: 3 circuits $\times$ 3 dtypes).

M Step-budget and step-size figures

Main paper Fig. 3 shows the cross-circuit step budget. Figs. 2 and 3 show the step-size and original step-budget ablation; they confirm T=100 and η=0.3 as robust defaults.

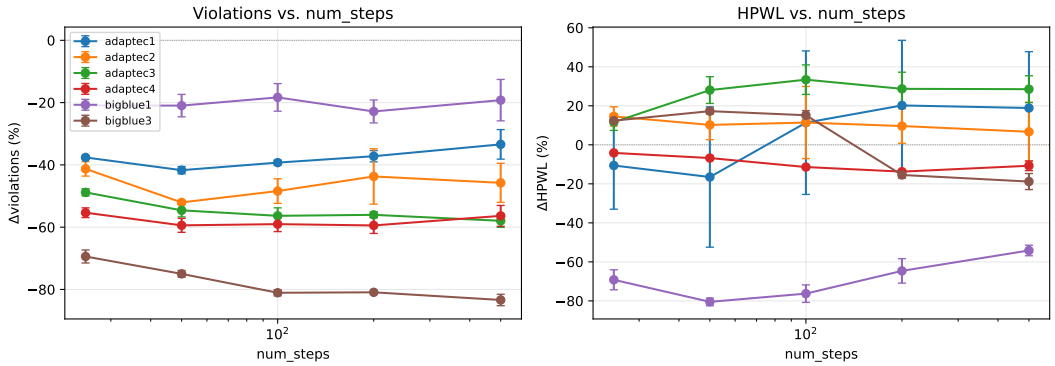

Figure 2: Number of repair steps ablation.

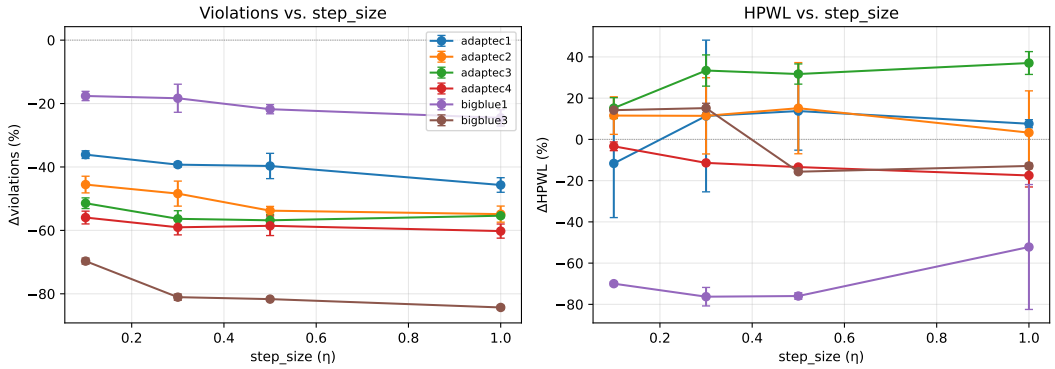

Figure 3: Step size ablation.

N Reproducibility and released artifacts

All code, configs, and result JSONs are released under the supplementary archive. The main-paper numbers reproduce from the following commands on a single A800 80 GB GPU (full sweep $\sim$ 70 min):

```
python scripts/run_main_neurips.py        # main result, ~30 min
python scripts/run_timestep_sweep.py      # timestep sweep, ~12 min
python scripts/run_mem_profile.py         # memory profile, ~16 min
python scripts/run_component_ablation.py  # component ablation, ~12 min
python scripts/compute_neurips_stats.py   # main + supplement tables
python scripts/analyze_extra_experiments.py  # lambda/step/failure
python scripts/analyze_component_ablation.py # supplement Table 3
```

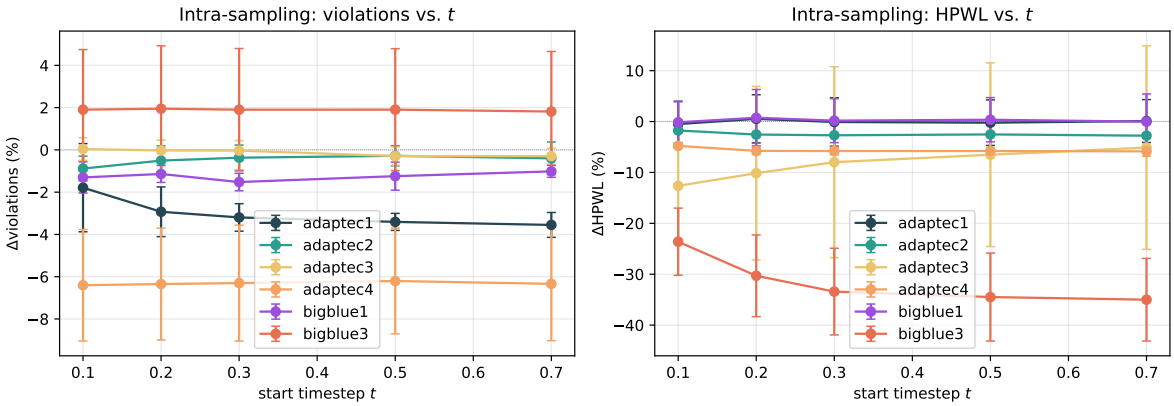

Figure 4: Intra-sampling HPWL change as a function of t_{start}, across 6 circuits and 3 seeds. Bigblue3 (the largest tractable circuit) shows a striking monotonic improvement, peaking at -34.5% HPWL. Smaller circuits show small effects, suggesting backbone-prior leverage scales with circuit size.

Seeds are fixed ({42, 123, 300, 2024} for the 4-seed runs; {42, 123, 300} for 3-seed sweeps). All result JSONs are under `results/ispd2005/` and `results/vsr_extra/`.